

方法:

本实验的目标是在长序列 self-attention 中，用 local dense block + zig-zag/expander cross-block sparse attention 替代完全图 attention。长度为 N 的 token 序列按 B 个 token 一组划分为 $q=N/B$ 个 block; block 是大图 G 的顶点，block 内端口/token 位置是小图 H 的顶点。需要注意的是，local dense block 本身不是 H ， H 只用于 zig-zag 跨块边的端口扰动。

每个 token 的注意力来源分为两类：一是同一 block 内 B 个 token 的 full attention；二是通过 zig-zag 产生的 d^2 条跨 block 边。理论上每行 raw $K = B + d^2$ ，attention pair 数由 N^2 降到约 $N(B+d^2)$ 。当前实现默认 non-causal encoder-style attention。

方法参数:

核心参数包括序列长度 N 、block 大小 B 、 H 的度数 d 、 G 的 rotation map、 H 的邻接结构、层数、 d_model 、head 数、FFN 维度、dropout、学习率和 seed。当前阶段主要固定 $B=16, d=2$ ，因此 sparse 方法的 raw K 为 $16+2^2=20$ ；dense 的 $K=N$ ，local-only 的 $K=16$ 。

当前实现中， H 使用 cycle-like 邻接： $d=2$ 时 $H(i)=\{(i+1) \bmod B, (i-1) \bmod B\}$ ； $d=3$ 时继续加入下一层偏移。 G 使用 cyclic port-labeled rotation map，而不是已经充分调参或理论最优的 expander G 。这个选择是 MVP 验证用的简单结构。

比较方法:

方法	作用	当前状态
Dense	完整 self-attention，作为小 N 质量上界和 reference path。	已运行
Local-only	只看同一 block 内 token，检验跨块边是否必要。	已运行
Local + zig-zag	local dense block 加 zig-zag 跨块边。	当前新方法，已运行
Random	local dense block 加 same-budget 随机跨块边，每个 token 跨块边数为 d^2 。	已运行，用于公平对比
BigBird/Longformer-style	相关 sparse attention baseline。	当前仅作参考，尚未进入结果表

使用模型:

当前 zig-zag 诊断主要使用 project-local 的 TinyTransformer，而不是完整 HuggingFace/LRA

训练栈。模型配置为 8 layers, d_model=128, 4 heads, ffn_dim=256, dropout=0.1, GELU。

输入表示为 token embedding + position embedding；每层采用 pre-LN 残差结构： $x = x + \text{Attention}(\text{LayerNorm}(x))$, $x = x + \text{FFN}(\text{LayerNorm}(x))$ 。最终分类头读取最后一个 token 的 hidden state。

任务：

当前真正形成阶段性证据的是 copy_first。输入为 $[x_0, x_1, \dots, x_{N-1}]$ ，模型只从最后一个 token 的表示做四分类，目标是输出 x_0 。

$x_i \in \{1, 2, 3, 4\}$ ，因此随机猜测准确率约为 25%。当 $N=1024, B=16$ 时， x_0 与最后一个 token 不在同一 block，local-only 不能直接看到 x_0 。

参数：

项目	当前设置 / 说明
主要运行	$N=1024$, $B=16$, $d=2$, steps=1000, batch_size=16, eval_batches=20, num_values=4, learning_rate=0.001, seeds=0/1/2。
注意力数	dense $K=1024$; local-only $K=16$; random 和 zig-zag raw $K=16+4=20$; $N=1024$ 时 attention_pair_count 为 20480。
H 连边	$d=2$ 时 $H(i)=\{(i+1) \bmod 16, (i-1) \bmod 16\}$ 。这是 cycle-neighbor 结构，不是强 expander 图。
G 连边	使用 cyclic Rot_G: $\text{offset}=(\text{port}/2)+1$ ；偶数 port 连到 $(v+\text{offset}) \bmod q$ 并翻转到 port^1 ，奇数 port 连到 $(v-\text{offset}) \bmod q$ 并翻转到 port^1 。
Mask 测试	覆盖 $N=64/128, B=8/16, d=2/3$; dense-mask、neighbor、split 和 blockpair 小规模数值误差小于 $1e-5$ 。

目前结果：

项目	结果
$N=256$ copy_first	dense、random、zig-zag 都能解，local-only 失败。
$N=512$ copy_first	zig-zag 能解；local-only 失败；

	random 在该 seed/config 下失败。
N=1024 copy_first	三 seed 中 zig-zag 全部达到 1.0 accuracy, 且都在 step 250 前收敛; local-only 全失败; dense seed 0/2 成功、seed 1 为 0.7812; random seed 0/1 失败、seed 2 成功。

目前问题:

以上为 codex 自主发挥，很有问题。

第一，copy_first 是人为构造的 synthetic gate，任务过于简单。目前有两种评测思路，使用合成任务或小参数 llm 评测。

第二，H 目前是 cycle-neighbor，G 是 cyclic rotation map，二者还没有经过系统搜索，也不是强 expander 图。后续如果要写成 zig-zag/expander 结论，需要补 G/H 消融。

第三，当前模型学习率、dropout、层数等超参没调。尽管要求 ai 从现有的代码中进行改进，但由于选取的仓库问题，实际和 from scratch 实现区别不大。

第四，zig-zag 在 N=1024 三个 seed 中比 dense/random 更早更稳定地收敛，但目前由于评测任务，暂时不能给出结论。

合成任务:

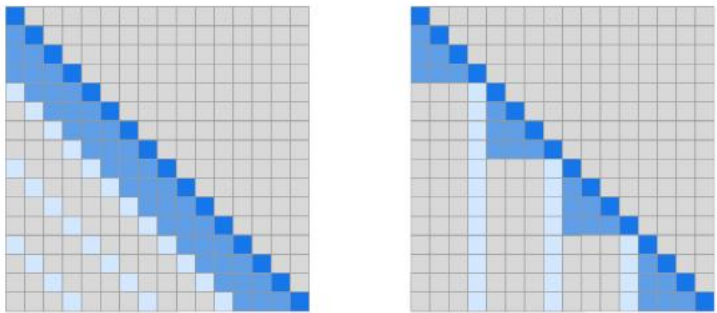
下表是后续可考虑的合成任务范围。associative_recall 只做了早期 smoke，其他任务尚未进入本阶段结果。

任务	来源状态	输入输出形式	测什么能力	设置
Copy	经典 copy 类任务; 本阶段实际使用 copy_first 变体。	输入一串 token, 模型从最后位置预测第一个 token。	基本信息保持和跨 block 长程传递。	vocab 4-128, train length 256-1024, 可 eval 到 2048。
Reverse	经典 algorithmic transduction; Universal Transformer 等工作使用过相近任务。	输入一串 token, 输出反序序列。	非局部位置映射。	vocab 32-128, 长度 64-1024。
Associative Recall	Neural Turing Machines 等使用的内 容寻址任务。	输入 key-value pairs, 再给 query key, 输出 value。	内容寻址和远距离检索。	pair 数 16-256; 当前早期配置尚不具结论性。

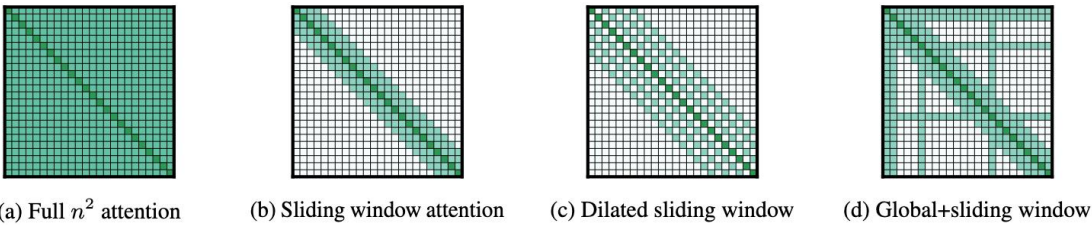
Selective Retrieval	接近 selective copying / RULER retrieval 类任务。	长噪声序列中插入 marker 和目标 token，输出目标。	从无关上下文中定位关键 token。	noise length 512-8192, marker 随机位置。
Needle-lite	低成本 NIAH/RULER 风格变体。	token 噪声里放一条 needle, 最后问 needle 内容。	长上下文精确检索。	小模型分类/生成目标 token, 不接 LLM。
Induction Head	Anthropic induction heads 与 Mamba 相关讨论中常见 pattern。	构造 A B ... A ?, 要求预测 B。	重复 pattern 和 in-context 关联。	vocab 128, 重复间隔 sweep。
括号匹配	Dyck language / bounded Dyck language 常用任务。	生成括号串, 预测是否合法或位置深度。	层级结构、栈式依赖。	最大深度 4-16, 长度 128-2048。

文献调研:

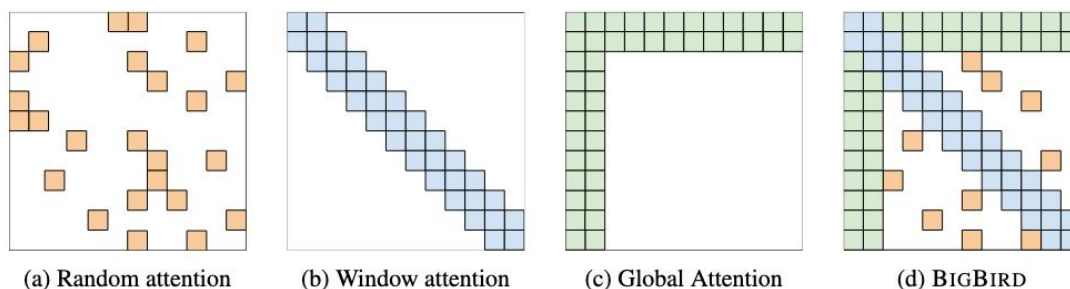
Sparse Transformer(2019): 核心是使用固定/分解的稀疏 attention pattern 降低长序列代价，启发本实验把 attention pair 从 N^2 控制到 $N \cdot K$ 。



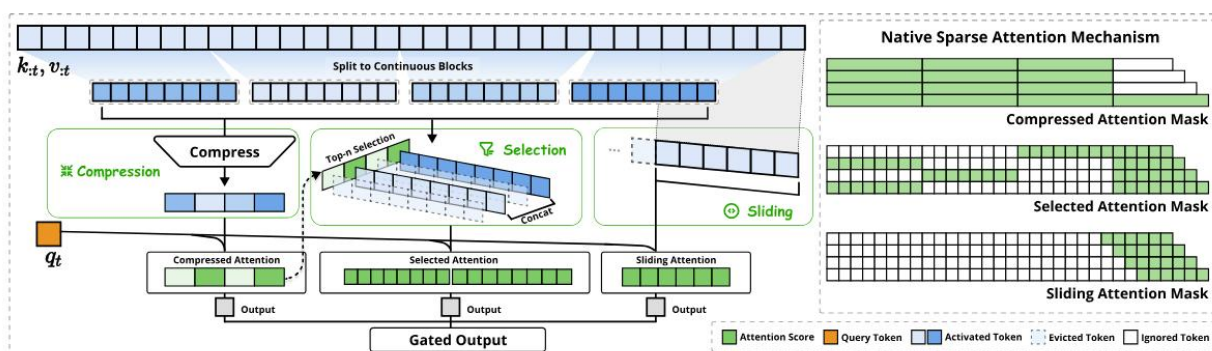
Longformer(2020): sliding window + global attention。它说明 local window 可以作为强工程基线，但长程信息通常还需要 global 或跨窗口机制。



BigBird(2020): local block + random block + global token。它是当前 random same-budget baseline 的主要参考之一。



Native Sparse Attention(2025, DeepSeek): 主要思路是分块压缩和选择性细粒度 attention，目标更偏向大模型长上下文效率。



Routing Transformer(2020)、Reformer(2020): 分别偏向 routing/clustering 和 LSH attention, 和当前 local block + zig-zag cross-edge 的结构不同，因此本阶段没有作为主 baseline。

总体观察：经典 sparse attention 的代表工作集中在 2019-2020 年左右，近年更多工作关注长上下文推理效率、KV cache 或分块选择机制。当前实验仍处在小模型 synthetic 验证阶段。